

4 - SIEM

Name of report: SIEM_LAB_4_Nikita_Niakhai

Course: Secure Development

Performed by: Nikita Niakhai

Date submission: 25.03.2026



This lab is designed to introduce students to security solutions, specifically a SIEM. Students can use any SIEM of choice — a solid recommendation is the open source platform **Wazuh**, which provides a fleet of capabilities at no cost.

In this lab, students will interact with additional tools such as **VirusTotal**, **YARA**, **osquery**, and **SOAR**, and gain experience with SIEM log analysis, vulnerability detection, and more.

Part A

Task 1 — Introduction

SIEM [1] [2] — (stands for Security Information and Event Management) is a security tool used to monitor, gather event information *from many sources), respond to attacks and alerts.

Features:

It gives ability to **collect, manage, gather and display** in a dashboard view with different filters data from the systems (applications, databases, OS, network devices, proxies).

- Logs information.
- Detects threats.
- Responds to incidents.
- Alerts SOC (security operation center) team.
- Filter using AI by event/source/type of event information.
- Gives ability to do Forensics analysis.
- Provides integrability with other security tools.

Examples and tools:

Proprietary: Microsoft Sentinel

a. SIEM Architecture

Wazuh consists of main components: wazuh-manager (server), wazuh-dashboard, wazuh-indexer and wazuh-agents/modules on end nodes.

Wazuh agents establish encrypted with AES TCP connection on port 1514 with Wazuh server and continuously send data about events.. Then Wazuh server uses defined Rules and Decoders to parse and “read” the data.

Then Wazuh server securely with TLS enabled sends data about alerts/warnings/events to Filebeat, which reads them and sends to Wazuh Indexer, after that Wazuh dashboard queries (user/password authentication, TLS) with API (port 55000/TCP) the Wazuh server to display the data.

In this lab I used All-in-One deployment (server-wazuh components are hosted on one VM), you can divide hosting all main components server, indexer, dashboard between different VMs, containers.

The diagram below represents a Wazuh deployment architecture. It shows how the Wazuh server and the Wazuh indexer nodes can be configured as clusters, providing load balancing and high availability.

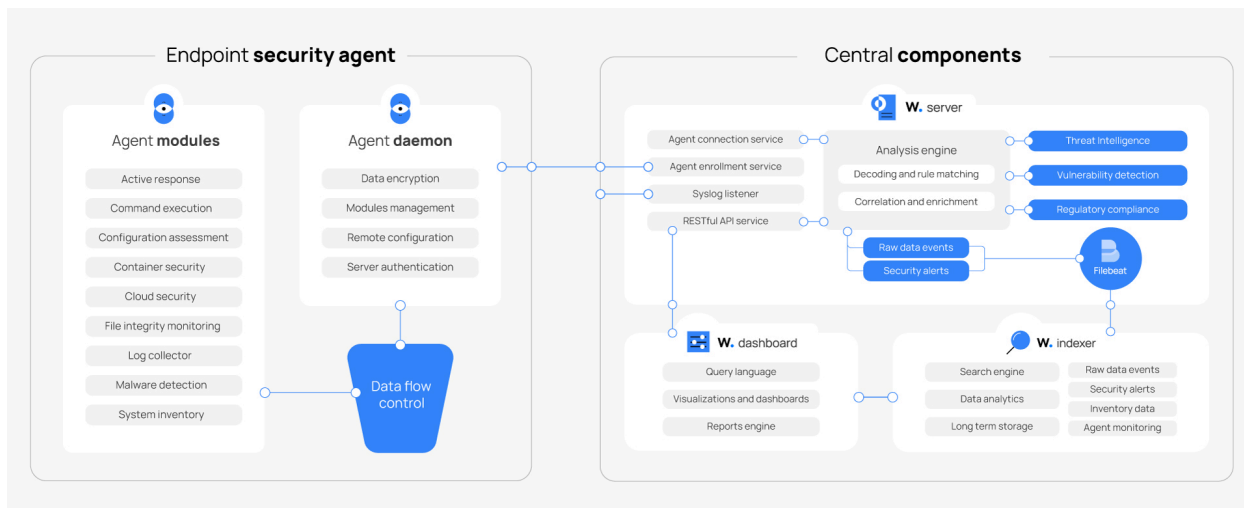


Figure 1.1 — SIEM architecture diagram

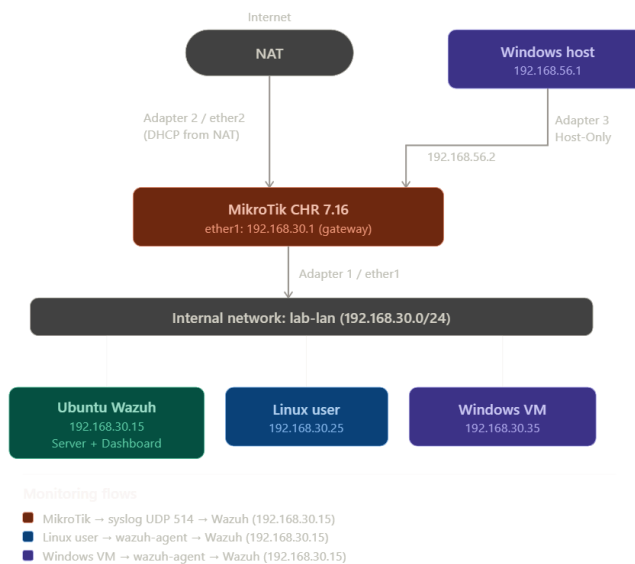


Figure 1.2 — Infrastructure diagram

Table for Infrastructure

VM/Service	VirtualBox adapter(s)	IP	Info
MikroTik CHR 7.16	Adapter 1: Internal Network lab-lan , Adapter 2: NAT Adapter 3: Host-Only Adapter → VirtualBox Host-Only Ethernet Adapter #1 (192.168.56.x)	ether1: DHCP (10.0.2.15), ether2: 192.168.30.1 ether2: 192.168.56.2	admin:admin

VM/Service	VirtualBox adapter(s)	IP	Info
Ubuntu Wazuh	Adapter 1: Internal Network lab-lan	192.168.30.15 (wazuh)	ubuntu:ubuntu
Linux user	Adapter 1: Internal Network lab-lan	192.168.30.25	ubuntu:ubuntu
Windows	Adapter 1: Internal Network lab-lan	192.168.30.35	user:user and admin:admin
Wazuh Dashboard			admin:9zq2AK79*8vOZ6ID?? +CHVDgcNMS31Sp

- Setting up communication on infrastructure

```
# MikroTik router configuration

# ether1 = LAN side
/ip address add address=192.168.30.1/24 interface=ether1

# ether2 = NAT adapter (internet via VirtualBox NAT)
/ip dhcp-client add interface=ether2 disabled=no

# Enable NAT masquerade so LAN hosts reach internet
/ip firewall nat add chain=srcnat out-interface=ether2 action=masquerade

# Allow DNS relay for LAN clients
/ip dns set allow-remote-requests=yes

# Verify internet works
/ping 8.8.8.8 count=3

# This puts MikroTik on the same subnet as your Windows Host-Only adapter
/ip address add address=192.168.56.2/24 interface=ether3

# allow forwarding between ether3(With Host) and ether1 (Lan)
/ip firewall filter add chain=forward in-interface=ether3 out-interface=ether1 action=accept
/ip firewall filter add chain=forward in-interface=ether1 out-interface=ether3 action=accept

# On host machine Windows:
route add 192.168.30.0 mask 255.255.255.0 192.168.56.2 metric 5

# Ubuntu Wazuh server

# Find your interface name
ip link show

# Set static IP via netplan
sudo tee /etc/netplan/01-static.yaml << 'EOF'
network:
  version: 2
  ethernets:
    enp0s3:
      addresses: [192.168.30.15/24]
      routes:
```

```

        - to: default
          via: 192.168.30.1
    nameservers:
      addresses: [192.168.30.1, 8.8.8.8]
EOF

sudo netplan apply

# Verify
ping -c 3 192.168.30.1    # gateway
ping -c 3 8.8.8.8        # internet
ping -c 3 google.com     # DNS

# Linux user machine

# Same approach, different IP
sudo tee /etc/netplan/01-static.yaml << 'EOF'
network:
  version: 2
  ethernets:
    enp0s3:
      addresses: [192.168.30.25/24]
      routes:
        - to: default
          via: 192.168.30.1
      nameservers:
        addresses: [192.168.30.1, 8.8.8.8]
EOF

sudo netplan apply
ping -c 3 192.168.30.1
ping -c 3 192.168.30.15

# Windows VM

# Find your adapter index
Get-NetAdapter

# Found that it is 4

# Set static IP
New-NetIPAddress -InterfaceIndex 4 -IPAddress 192.168.30.35 -PrefixLength 24 -DefaultGate
way 192.168.30.1
Set-DnsClientServerAddress -InterfaceIndex 4 -ServerAddresses 192.168.30.1,8.8.8.8

# Verify
ping 192.168.30.1
ping 192.168.30.15
ping 8.8.8.8
` ``

```

Replace `InterfaceIndex 6` with the actual index from `Get-NetAdapter`.

Step 6 – Verify full connectivity from every machine

Every VM should be able to reach every other VM and the internet:

```

```
ping 192.168.30.1 # MikroTik
ping 192.168.30.15 # Wazuh
ping 192.168.30.25 # Linux user
ping 192.168.30.35 # Windows
ping 8.8.8.8 # Internet
```

```
[admin@MikroTik] > /ip address add address=192.168.30.1/24 interface=ether1
[admin@MikroTik] > /ip dhcp-client add interface=ether2 disabled=no
[admin@MikroTik] > /ip firewall nat add chain=srcnat out-interface=ether2 action
masquerade
[admin@MikroTik] > /ip dns set allow-remote-requests=yes
[admin@MikroTik] > /ping 8.8.8.8
 SEQ HOST SIZE TTL TIME STATUS

 0 8.8.8.8 56 255 16ms201us
 1 8.8.8.8 56 255 1ms339us
 2 8.8.8.8 56 255 1ms58us
 sent=3 received=3 packet-loss=0% min-rtt=1ms58us avg-rtt=6ms199us
 max-rtt=16ms201us

[admin@MikroTik] > /ping google.com
 SEQ HOST SIZE TTL TIME STATUS

 0 216.239.38.120 56 255 1ms20us
 1 216.239.38.120 56 255 972us
 2 216.239.38.120 56 255 1ms195us
 sent=3 received=3 packet-loss=0% min-rtt=972us avg-rtt=1ms62us
 max-rtt=1ms195us
```

Figure — Mikrotik communication setup and test via `ping`

```
linux-machine [Running] - Oracle VirtualBox
File Machine View Input Devices Help
root@linux-machine:~# cat /etc/netplan/01-static.yaml
network:
 version: 2
 ethernet:
 enp0s3:
 addresses: [192.168.30.25/24]
 routes:
 - to: default
 via: 192.168.30.1
 nameservers:
 addresses: [192.168.30.1, 8.8.8.8]
root@linux-machine:~# ping 192.168.30.1
PING 192.168.30.1 (192.168.30.1) 56(84) bytes of data.
64 bytes from 192.168.30.1: icmp_seq=1 ttl=64 time=18.5 ms
64 bytes from 192.168.30.1: icmp_seq=2 ttl=64 time=6.88 ms
64 bytes from 192.168.30.1: icmp_seq=3 ttl=64 time=13.3 ms
^C
--- 192.168.30.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 6.878/12.903/18.529/4.764 ms
root@linux-machine:~# ping 192.168.30.15
PING 192.168.30.15 (192.168.30.15) 56(84) bytes of data.
64 bytes from 192.168.30.15: icmp_seq=1 ttl=64 time=3.05 ms
64 bytes from 192.168.30.15: icmp_seq=2 ttl=64 time=1.18 ms
64 bytes from 192.168.30.15: icmp_seq=3 ttl=64 time=1.17 ms
^C
--- 192.168.30.15 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.172/1.799/3.046/0.881 ms
root@linux-machine:~# ping google.com
PING google.com (74.125.205.138) 56(84) bytes of data.
64 bytes from le-in-f138.1e100.net (74.125.205.138): icmp_seq=1 ttl=254 time=20.0 ms
64 bytes from le-in-f138.1e100.net (74.125.205.138): icmp_seq=2 ttl=254 time=62.2 ms
64 bytes from le-in-f138.1e100.net (74.125.205.138): icmp_seq=3 ttl=254 time=42.6 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 20.042/41.608/62.184/17.218 ms
root@linux-machine:~#
```

Figure — Linux machine `ping` test (windows, wazuh, router, nat)

```
Ubuntu Controller (Clean Ubuntu Desktop) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Mar 22 15:07

root@UbuntuDesktop: ~
root@UbuntuDesktop:~# cat /etc/netplan/01-static.yaml
network:
 version: 2
 ethernets:
 enp0s3:
 addresses: [192.168.30.15/24]
 routes:
 - to: default
 via: 192.168.30.1/24
 nameservers:
 addresses: [192.168.30.1, 8.8.8.8]
root@UbuntuDesktop:~# ping 192.168.30.1
PING 192.168.30.1 (192.168.30.1) 56(84) bytes of data:
64 bytes from 192.168.30.1: icmp_seq=1 ttl=64 time=18.2 ms
64 bytes from 192.168.30.1: icmp_seq=2 ttl=64 time=7.24 ms
64 bytes from 192.168.30.1: icmp_seq=3 ttl=64 time=3.89 ms
^C
--- 192.168.30.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 3.887/9.771/18.190/6.108 ms
root@UbuntuDesktop:~# ping google.com
PING forcesafesearch.google.com (216.239.38.120) 56(84) bytes of data:
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=1 ttl=
8 ms
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=2 ttl=
3 ms
^C
--- forcesafesearch.google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 2.181/14.219/26.257/12.038 ms
root@UbuntuDesktop:~#
```

Figure — Wazuh machine `ping` test (windows,linux machine, router, nat)

```
Windows10 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Select Administrator: Windows PowerShell

PS C:\Windows\system32> New-NetIPAddress -InterfaceIndex 4 -IPAddress 192.168.30.35 -PrefixLength 24 -DefaultGateway 192.168.30.1

IPAddress : 192.168.30.35
InterfaceIndex : 4
InterfaceAlias : Ethernet
AddressFamily : IPv4
Type : Unicast
PrefixLength : 24
PrefixOrigin : Manual
SourceOrigin : Manual
AddressState : Tentative
ValidLifetime : Infinite ((TimeSpan)::MaxValue)
PreferredLifetime : Infinite ((TimeSpan)::MaxValue)
SkipSource : False
PolicyStore : ActiveStore

IPAddress : 192.168.30.35
InterfaceIndex : 4
InterfaceAlias : Ethernet
AddressFamily : IPv4
Type : Unicast
PrefixLength : 24
PrefixOrigin : Manual
SourceOrigin : Manual
AddressState : Invalid
ValidLifetime : Infinite ((TimeSpan)::MaxValue)
PreferredLifetime : Infinite ((TimeSpan)::MaxValue)
SkipSource : False
PolicyStore : PersistentStore
```

Figure — Windows machine setting up gateway

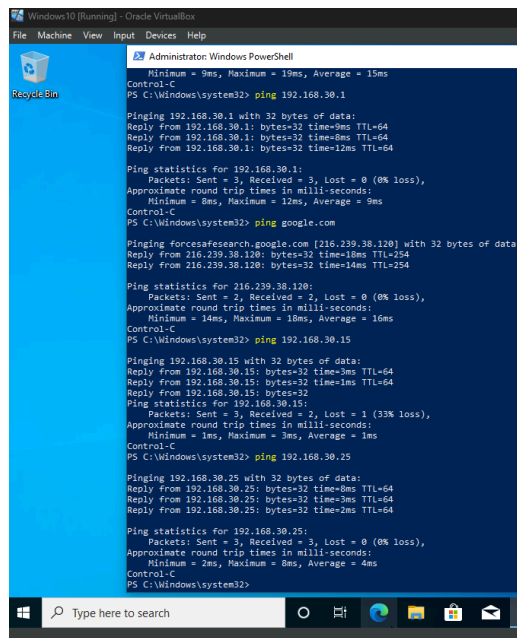


Figure — Wazuh machine `ping` test (linux machine, wazuh, router, nat)

- Installed wazuh server on wazuh machine [\[installation guide\]](#)

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh
-a

sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt

sed -i "s/^deb /#deb /" /etc/apt/sources.list.d/wazuh.list
apt update
```

```
23/03/2026 11:03:55 INFO: Wazuh manager vulnerability detection configuration finished.
23/03/2026 11:03:55 INFO: Starting service wazuh-manager.
23/03/2026 11:04:10 INFO: wazuh-manager service started.
23/03/2026 11:04:10 INFO: Starting Filebeat installation.
23/03/2026 11:05:14 INFO: Filebeat installation finished.
23/03/2026 11:05:16 INFO: Filebeat post-install configuration finished.
23/03/2026 11:05:16 INFO: Starting service filebeat.
23/03/2026 11:05:19 INFO: filebeat service started.
23/03/2026 11:05:19 INFO: --- Wazuh dashboard ---
23/03/2026 11:05:19 INFO: Starting Wazuh dashboard installation.
23/03/2026 11:13:22 INFO: Wazuh dashboard installation finished.
23/03/2026 11:13:22 INFO: Wazuh dashboard post-install configuration finished.
23/03/2026 11:13:22 INFO: Starting service wazuh-dashboard.
23/03/2026 11:13:23 INFO: wazuh-dashboard service started.
23/03/2026 11:13:25 INFO: Updating the internal users.
23/03/2026 11:13:32 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.
23/03/2026 11:13:51 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and password.
23/03/2026 11:14:28 INFO: Initializing Wazuh dashboard web application.
23/03/2026 11:14:28 INFO: Wazuh dashboard web application initialized.
23/03/2026 11:14:28 INFO: --- Summary ---
23/03/2026 11:14:28 INFO: You can access the web interface https://wazuh-dashboard-ip:443
User: admin
Password: 92z2AK79+0s0261D7?+CHU0gcMS31Sp
23/03/2026 11:14:28 INFO: Installation finished.
```

Figure — Wazuh installation is complete



```

root@Linux-Wazuh:~# ./media/setup-wazuh.sh
wazuh-install-files/wazuh-passwords.txt
Admin user for the web user interface and Wazuh indexer. Use this user to log in to Wazuh dashboard
indexer_username: 'admin'
indexer_password: '9zq2AK79+6u0261D??+CHU0gcNMS313p'

Anomaly detection user for the web user interface
indexer_username: 'anomalyadmin'
indexer_password: '89o7wYICj7r15EXsxxq3pefhqd4N0n.c'

Wazuh dashboard user for establishing the connection with Wazuh indexer
indexer_username: 'kibanaserver'
indexer_password: 'ry14FRdc286f79n5B7p9m0aFobKigK2'

Regular Dashboard user, only has read permissions to all indices and all permissions on the .kibana index
indexer_username: 'kibana'
indexer_password: 'QLN9L15B3G6S.S.7c2Uy9ulfJw.07C24'

Filebeat user for CRUD operations on Wazuh indices
indexer_username: 'logstash'
indexer_password: 'iqeTE8H0p==p783gg2y0zu600+s.j24f'

User with READ access to all indices
indexer_username: 'readall'
indexer_password: 'Pcu.1z?.z2L4.34b6G8u0FUYtiBUriYa'

User with permissions to perform snapshot and restore operations
indexer_username: 'snapshotrestore'
indexer_password: 'E6G6pYTRm?.ak1Bb70u61JhKt5JB12P'

Password for wazuh API user
api_username: 'wazuh'
api_password: '0=114SS=0n.j2Jyu0o5K4SFf1ZFhzf1'

Password for wazuh-uci API user
api_username: 'wazuh-uci'
api_password: '3su2RU8H10e9muref6hc2e0m7d0b12'

```

Figure — Wazuh credentials querying

## b. Open Source Advantages & Business Model

I researched those solutions [3 about wazuh], [4 good russian source], [5] and referenced lecture slides. What is HIDS and NIDS [6]. Overview on SOC tools [7]. Business model is described [here](#).

### Advantage 1: Cost Savings

Open source SIEM eliminates licensing fees, allowing budget-conscious teams to deploy powerful tools like log management and alerting without upfront costs. This is ideal for startups or SMBs testing security features before scaling.

### Advantage 2: Customization

Users can modify the codebase to fit specific needs, such as integrating HIDS/NIDS or adapting to unique environments like cloud or on-premises setups. This flexibility supports tailored compliance and threat hunting in SOCs.

### Advantage 3: Well review and documented, Community

Active communities drive rapid updates, bug fixes, and shared enhancements, providing transparency and reducing vendor lock-in risks. For solutions like Wazuh, this means free forums, documentation, and contributions from thousands of users

### Q: How do open source vendors make money?

Open source SIEM vendors monetize through paid professional support (e.g., 24/7 help desks), enterprise subscriptions for advanced features or scalability (e.g., Wazuh Cloud tiers from \$16/month), managed cloud hosting, and partner programs offering discounts on consulting or certified deployments. Dual licensing or "open core" models add premium modules, while services like training and integrations generate revenue without charging for the core software.

## Task 2 — Setup Infrastructure

### a. Configure SIEM with 3+ Unique Devices

#### Install wazuh agents on VMs

References: [\[mikrotik setup\]](#), [\[windows agent\]](#), [\[linux agent\]](#).

```
Install Wazuh agent on Linux user
```

```
#Install the following packages if missing:
```

```

apt-get install gnupg apt-transport-https

#Install the GPG key:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring --keyring gn
upg-ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644 /usr/share/keyrings/wazuh.gpg

#Add the repository:
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stab
le main" | tee -a /etc/apt/sources.list.d/wazuh.list

#Update the package information:
apt-get update

Select your package manager and run the command below. Replace the WAZUH_MANAGER value wit
h your Wazuh manager IP address or hostname:
WAZUH_MANAGER="192.168.30.15" apt-get install wazuh-agent

Enable and start the Wazuh agent service.
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent

Install Wazuh agent on Windows

Download the Windows installer to start the installation process.
https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.4-1.msi

run installation (Powershell)
.\wazuh-agent-4.14.4-1.msi /q WAZUH_MANAGER="192.168.30.15"

Start (Powershell)
Start-Service wazuhsvc

Files are stored C:\Program Files (x86)\ossec-agent

Send all logs to Wazuh via syslog (UDP 514)
/system logging action set remote address=192.168.30.15 remote-port=514

/system logging add topics=info action=remote
/system logging add topics=error action=remote
/system logging add topics=warning action=remote
/system logging add topics=critical action=remote
/system logging add topics=firewall action=remote

```

## Configuring: MikroTik to send syslog to Wazuh Server

- Paste the following code snippet at the bottom of the `/etc/rsyslog.conf` file on the Ubuntu endpoint. This code enables the UDP port **514** to listen for syslogs and adds a location to store the security events. Replaced `<YOUR_MIKROTIK_IP_ADDRESS>` with my IP address of the MikroTik device.

```

root@Linux-Wazuh:/media# tail -n 7 /etc/rsyslog.conf
$IncludeConfig /etc/rsyslog.d/*.conf

$ModLoad imudp
$UDPServerRun 514

if $fromhost-ip starts with '192.168.30.1' then /var/log/mikrotik.log
~

root@Linux-Wazuh:/media#

root@Linux-Wazuh:/media# touch /var/log/mikrotik.log
root@Linux-Wazuh:/media# ls -l /var/log/mikrotik.log
-rw-r--r-- 1 syslog adm 547 Mar 23 14:16 /var/log/mikrotik.log
root@Linux-Wazuh:/media# systemctl restart rsyslog
root@Linux-Wazuh:/media# systemctl status rsyslog

```

Figure — `/etc/rsyslog.conf` file with new configurations and log file is created

- Create the `mikrotik.log` file in the `/var/log` directory to store the syslog events.
- Change the file ownership for the `/var/log/mikrotik.log` file to `syslog` and group to `adm`.
- Restart the `rsyslog` utility for the changes to take effect.

```

root@Linux-Wazuh:~# touch /var/log/mikrotik.log
root@Linux-Wazuh:~# chown syslog:adm /var/log/mikrotik.log
root@Linux-Wazuh:~# systemctl restart rsyslog

```

- On the Wazuh server, edit `/var/ossec/etc/ossec.conf` and add inside the `<ossec_config>` block:
  - `allowed-ips` specifies which endpoints are allowed to send logs to this server. Replace `<CIDR_NOTATION>` with the CIDR address of the network on which your endpoints are. If you monitor just one endpoint, you can put its IP address.
  - `local-ip` specifies the local IP address of the Wazuh server to be used for this connection. Replace `<WAZUH_MANAGER_IP>` with the IP address of the Wazuh server.

```

<remote>
 <connection>syslog</connection>
 <port>514</port>
 <protocol>udp</protocol>
 <allowed-ips>192.168.30.1</allowed-ips>
 <local_ip>192.168.30.15</local_ip>
</remote>

```

```

Linux-Wazuh [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 7.2 /var/ossec/etc/ossec.conf
<remote>
 <connection>syslog</connection>
 <port>514</port>
 <protocol>udp</protocol>
 <allowed-ips>0.0.0.0</allowed-ips>
 <local_ip>192.168.30.15</local_ip>
</remote>

<remote>
 <connection>secure</connection>
 <port>1514</port>
 <protocol>tcp</protocol>
 <queue_size>131072</queue_size>
</remote>

```

Figure — `/var/ossec/etc/ossec.conf` new block is added

- Setup mikrotik to allow WinBox UI

```
[admin@MikroTik] > /ip service enable winbox
[admin@MikroTik] > /ip firewall filter add chain=input protocol=tcp dst-port=829
1 action=accept place-before=0
```

- Install WinBox on my host machine and connect to edit Mikrotik configurations.
  - SW already found to which endpoint to connect

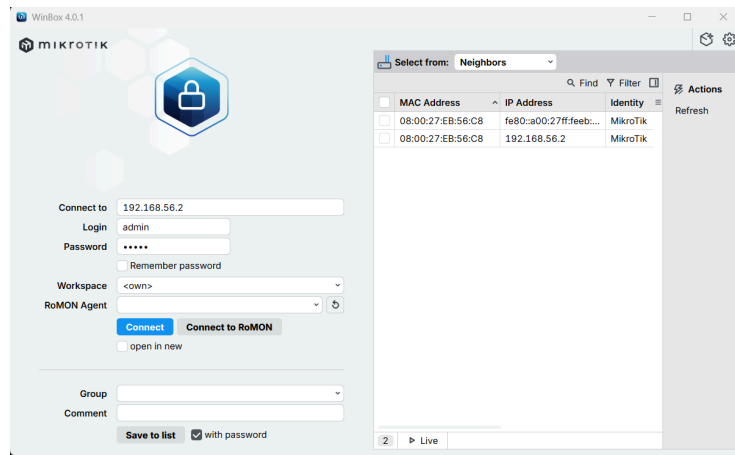


Figure — WinBox logging to mikrotik router

- Open Logging tab

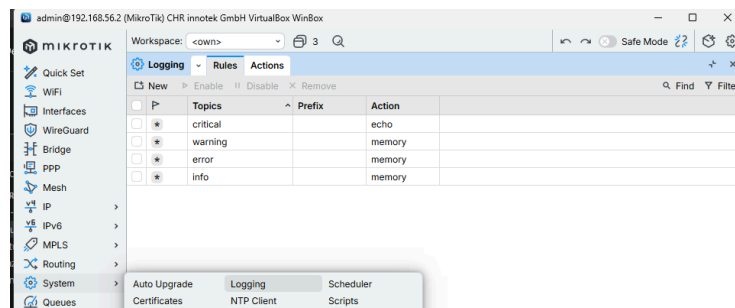


Figure — Inside mikrotik configuration for allowing forwarding to remote node

- Click on the **System** > **Logging** > **Rules** tab and create new or modify the existing rules where the topic's **error**, **info**, and **warning** are set to **remote**.

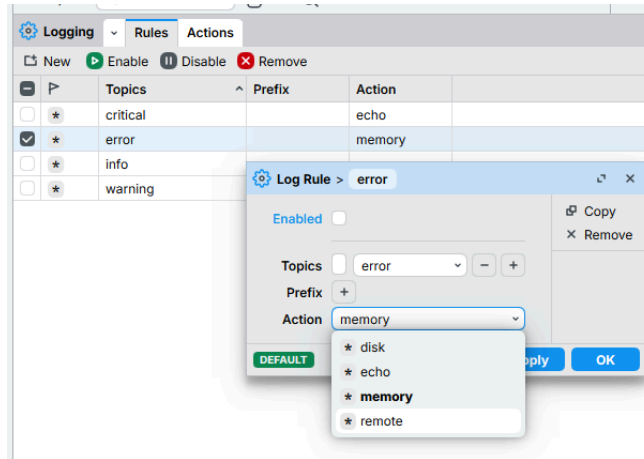


Figure — Changing rules to remote

| Logging                             |            |         |        |
|-------------------------------------|------------|---------|--------|
| Rules                               |            |         |        |
| Actions                             |            |         |        |
| New                                 | Enable     | Disable | Remove |
|                                     | Topics     | Prefix  | Action |
| <input type="checkbox"/>            | * critical |         | echo   |
| <input checked="" type="checkbox"/> | * error    |         | memory |
| <input type="checkbox"/>            | * info     |         |        |
| <input type="checkbox"/>            | * warning  |         |        |

Figure — All rules are changed to remote

- Click the **System > Logging > Actions** tab to edit the **remote** action. Set the **Remote address** field to the IP address of the Wazuh manager and the **Remote port** field to the Rsyslog listening port.

Log Action > remote

Name

remote

Type

remote

Remote Address

192.168.30.15

Remote Port

514

Src. Address

+

BSD Syslog

☒

Syslog Facility

daemon

Syslog Severity

emergency

Cancel

Apply

OK

Figure — configuring remote action to actually point to my Wazuh Server

- Setting up Wazuh Decoders from [documentation](#).
- Wrote 2 files, send them to wazuh server
- Restart `sudo systemctl restart wazuh-manager`
- Content of the files:

```
/var/ossec/etc/decoders/mikrotik_decoders.xml
```

```

<decoder name="mikrotik">
 <prematch>^Router0S7.1-logs: </prematch>
</decoder>

<decoder name="mikrotik1">
 <parent>mikrotik</parent>
 <regex type="pcre2">\S+ (\w+ \d+ \d+:\d+:\d+) MikroTik user (\S+) (.*) from (\d+.\d+.\d+.\d+) via (\w+)</regex>
 <order>logtimestamp, logged_user, action, ip_address, protocol</order>
</decoder>

<decoder name="mikrotik1">
 <parent>mikrotik</parent>
 <regex type="pcre2">\S+ (\w+ \d+ \d+:\d+:\d+) MikroTik dhcp-client on (\S+) (.*) address (\d+.\d+.\d+.\d+)</regex>
 <order>logtimestamp, interface, action, ip_address</order>
</decoder>

<decoder name="mikrotik1">
 <parent>mikrotik</parent>
 <regex type="pcre2">\S+ (\w+ \d+ \d+:\d+:\d+) MikroTik router (\S+)</regex>
 <order>logtimestamp, action</order>
</decoder>

```

```
var/ossec/etc/rules/mikrotik_rules.xml
```

```

<group name="Mikrotik,">

 <rule id="110000" level="0">
 <decoded_as>mikrotik</decoded_as>
 <description>Mikrotik-Event</description>
 </rule>

 <rule id="110001" level="5">
 <if_sid>110000</if_sid>
 <match>dhcp-client on ether</match>
 <description>MikroTik dhcp-client received an IP address $(ip_address)</description>
 </rule>

 <rule id="110002" level="5">
 <if_sid>110000</if_sid>
 <match>rebooted</match>
 <description>MikroTik router rebooted</description>
 </rule>

 <rule id="110003" level="5">
 <if_sid>110000</if_sid>
 <match>logged out from</match>
 <description>MikroTik user logged out via $(protocol)</description>
 </rule>

```

```
<rule id="110004" level="5">
 <if_sid>110000</if_sid>
 <match>logged in from</match>
 <description>MikroTik user logged in from $(ip_address) via $(protocol)</description>
</rule>

</group>
```

```
root@Linux-Wazuh:/media# cp mikrotik_decoders.xml /var/ossec/etc/decoders/
root@Linux-Wazuh:/media# cp mikrotik_rules.xml /var/ossec/etc/rules/
root@Linux-Wazuh:/media# cp mikrotik_rules.xml /var/ossec/etc/rules/
```

Figure — Copying created rules and decoders files to wazuh server

- Custom Rules and Decoders are seen in the dashboard, after the reboot of wazuh-manager.

Rules (6)

From here you can manage your rules.

relative\_dirname=etc/rules

WQL Custom rules

ID ↑	Description	Groups	Regulatory compliance	Level	File	Path
100001	sshd: authentication failed from IP 1.1.1.1.	authentication_failed, local, syslog, sshd	PCI DSS	5	local_rules.xml	etc/rules
110000	MikroTik-Event	Mikrotik		0	mikrotik_rules.xml	etc/rules
110001	MikroTik dhcp-client received an IP address <a href="#">ip_address</a>	Mikrotik		5	mikrotik_rules.xml	etc/rules
110002	MikroTik router rebooted	Mikrotik		5	mikrotik_rules.xml	etc/rules
110003	MikroTik user logged out via <a href="#">protocol</a>	Mikrotik		5	mikrotik_rules.xml	etc/rules
110004	MikroTik user logged in from <a href="#">ip_address</a> via <a href="#">protocol</a>	Mikrotik		5	mikrotik_rules.xml	etc/rules

Rows per page: 10

Figure — Verifying that custom rules are seen and applied in the SIEM architecture

Decoders (5)

From here you can manage your decoders.

relative\_dirname=etc/decoders

WQL Custom decoders

Name	Program name	Order	File ↑	Path
local_decoder_example	("pattern":"local_decoder_example")		local_decoder.xml	etc/decoders
mikrotik			mikrotik_decoders.xml	etc/decoders
mikrotik1		logtimestamp, logged_user, action, ip_address, protocol	mikrotik_decoders.xml	etc/decoders
mikrotik1		logtimestamp, interface, action, ip_address	mikrotik_decoders.xml	etc/decoders
mikrotik1		logtimestamp, action	mikrotik_decoders.xml	etc/decoders

Rows per page: 10

Figure — Verifying that custom decoders are seen and applied in the SIEM architecture

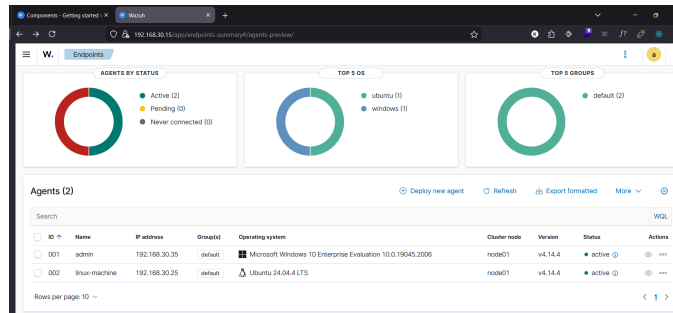


Figure 2.1 — SIEM dashboard showing all connected agents/devices

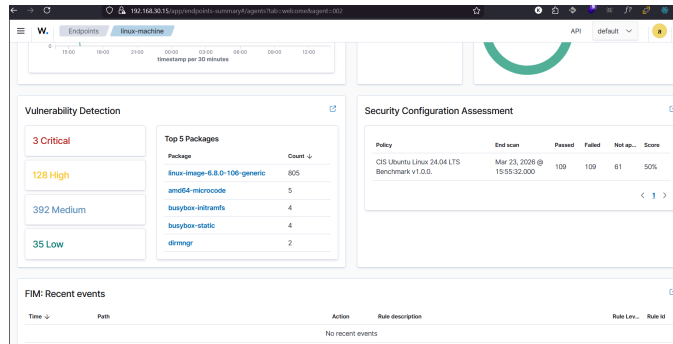


Figure 2.2 — Log data visible from Device 1 (e.g. Linux)

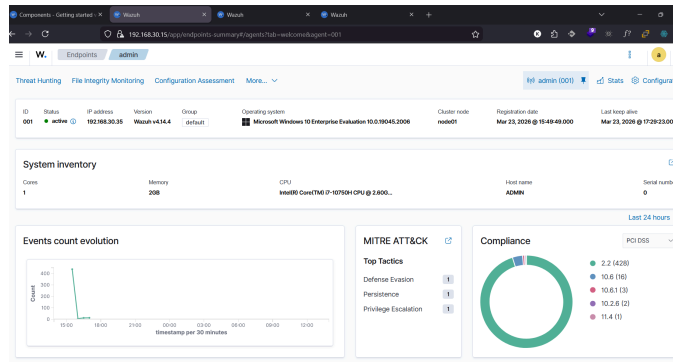


Figure 2.3 — Log data visible from Device 2 (e.g. Windows)



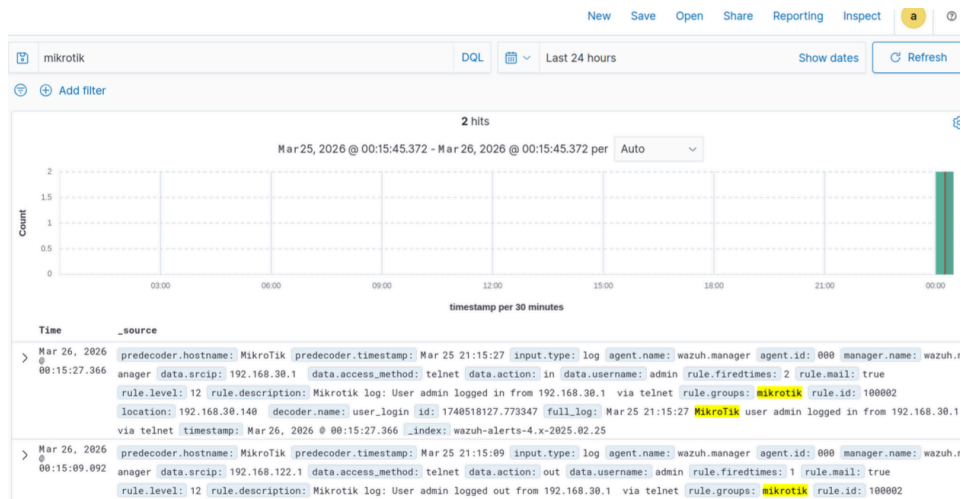


Figure 2.4 — Log data visible from Device 3 (e.g. Network device)

## b. Log Explanation — Why Can You View These Logs?

### Log 1 — Creating file with touch inside `/etc` on Linux-Wazuh machine

Log was triggered on PAM's process: that login session was opened successfully started (I intentionally `ctrl+D`).

This is type of data log (`input.type:`)

Name of agents `Linux-Wazuh`

Command executed is seen bellow attribute `data.command`: `/usr/bin/tee /etc/newfile.txt`

Rules attributes pointing to specific rule are part of this set of attributes:

```
rule.level:
 3
rule.pci_dss:
 10.2.5
rule.hipaa:
 164.312.b
rule.tsc:
 CC6.8, CC7.2, CC7.3
rule.description:
```

```
Linux-Wazuh [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@Linux-Wazuh:~# echo "hi!" | sudo tee /etc/newfile.txt
hi!
root@Linux-Wazuh:~#
root@Linux-Wazuh:~#
```

Figure — triggering log to appear

```

> Mar 24, 2026 @ 15:00:32.980 [predecoder.hostname]: Linux-Mazuh [predecoder.program_name]: sudo [predecoder.timestamp]: Mar 24 12:00:32 [input.type]: log
agent.name: Linux-Mazuh agent.id: 000 [manager.name]: Linux-Mazuh [data.srcuser]: root [data.dstuser]: root [data.tty]: pts/0
[data.pwd]: /root [data.command]: /usr/bin/tee /etc/newfile.txt [rule.mail]: false [rule.level]: 3 [rule.pci_dss]: 10.2.5, 1
0.2.2 [rule.hipaa]: 164.312.b [rule.tsc]: CC6.6, CC7.2, CC7.3 [rule.description]: Successful sudo to ROOT executed.
[rule.groups]: syslog, sudo [rule.mist_800_53]: AU.14, AC.7, AC.6 [rule.gdpr]: IV.32.2 [rule.firetime]: 1

```

Figure 2.5 — Selected Log 1 detail in SIEM

## Log 2 — Creating file with touch inside `/etc` on linux-machine

Event is triggered by me nano the file inside etc directory.

```

> Mar 24, 2026 @ 15:25:13.894 [predecoder.hostname]: linux-machine [predecoder.program_name]: sudo [predecoder.timestamp]: Mar 24 12:25:12 [input.type]: log
agent.ip: 192.168.30.25 agent.name: linux-machine agent.id: 002 [manager.name]: Linux-Mazuh [data.srcuser]: ubuntu
[data.dstuser]: root [data.tty]: tty1 [data.pwd]: /home/ubuntu [data.command]: /usr/bin/nano /etc/newfile.txt [rule.mail]: false
[rule.level]: 3 [rule.pci_dss]: 10.2.5, 10.2.2 [rule.hipaa]: 164.312.b [rule.tsc]: CC6.6, CC7.2, CC7.3 [rule.description]: Successful sudo to ROOT executed.
[rule.groups]: syslog, sudo [rule.mist_800_53]: AU.14, AC.7, AC.6 [rule.gdpr]: IV.32.2

```

Figure 2.6 — Selected Log 2 detail in SIEM

## Task 3 — Use Cases (B, C)

### Use Case B — Brute Force Attack Detection

#### Detecting a brute-force attack - Proof of Concept guide

User manual, installation and configuration guides. Learn how to get the most out of the Wazuh platform.

<https://documentation.wazuh.com/current/proof-of-concept-guide/detect-brute-force-attack.html>

wazuh.docs.



Describe the attack simulation tool used (e.g. Hydra, Medusa), target service (SSH, RDP, HTTP), and how detection rules are configured in the SIEM.

```

Example: simulate SSH brute force with Hydra
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://<target-ip> -t 4

Check SIEM rule triggering for brute force (Wazuh rule 5551, 5712, etc.)

```

```

root@Linux-Mazuh:~# hydra -l badguy -P /usr/share/wordlists/rockyou.txt ssh://192.168.30.25 -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret ser
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-24 12:59:33
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (1:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.30.25:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-24 12:59:37

```

```

root@Linux-Mazuh:~# hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.30.25 -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-24 12:42:25
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (1:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.30.25:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-24 12:42:27
root@Linux-Mazuh:~#

```

Figure 3b.1 — Brute force attack launched from attacker machine

```

W. Threat hunting 1 hit
Mar 23, 2026 @ 15:48:05.847 - Mar 24, 2026 @ 15:48:05.847
Export Formatted Reset view 674 available fields Columns Density 1 fields sorted Exit full screen
Timestamp agent_name rule.description rule.level rule.id
Mar 24, 2026 @ 15:42:28.5... linux-machine sshd authentication failed. 5 5760

```

Figure 3b.2 — SIEM alert: brute force detected on Device 1

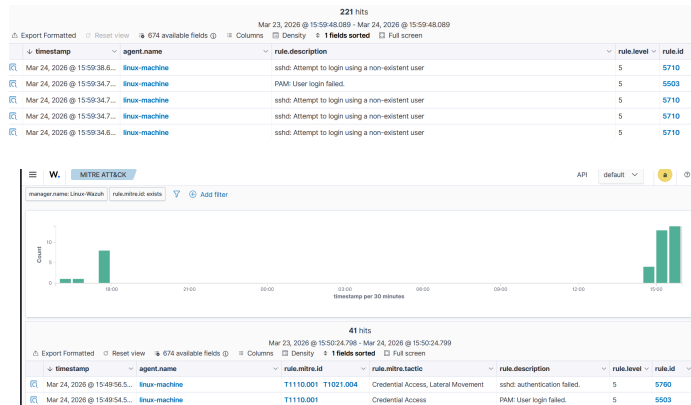


Figure 3b.3 — SIEM alert: brute force detected on Device 1

## Use Case C — CVE / Vulnerability Detection and Remediation

Vulnerability detection - Proof of Concept guide · Wazuh documentation

User manual, installation and configuration guides. Learn how to get the most out of the Wazuh platform.

<https://documentation.wazuh.com/current/proof-of-concept-guide/poc-vulnerability-detection.html>

wazuh.docs.



- Performed the following steps on the Wazuh server to confirm that the Wazuh Vulnerability Detection module is enabled and properly configured.
  - Check Vulnerability detection is enabled in Wazuh configuration `/var/ossec/etc/ossec.conf`

```
Linux-Wazuh [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 7.2 /var/ossec/etc/ossec.conf
<vulnerability-detection>
<enabled>yes</enabled>
<index-status>yes</index-status>
<feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>
```

Figure — `/var/ossec/etc/ossec.conf`

- Check for host ip address and certificates match the ones inside `filebeat.yml`

```
<indexer>
<enabled>yes</enabled>
<hosts>
<host>https://127.0.0.1:9200</host>
</hosts>
<ssl>
<certificate_authorities>
<ca>/etc/filebeat/certs/root-ca.pem</ca>
</certificate_authorities>
<certificate>/etc/filebeat/certs/wazuh-server.pem</certificate>
<key>/etc/filebeat/certs/wazuh-server-key.pem</key>
</ssl>
</indexer>
```

Figure — `/var/ossec/etc/ossec.conf`

```
GNU nano 7.2 /etc/filebeat/filebeat.yml
Wazuh - Filebeat configuration file
output.elasticsearch:
 - 127.0.0.1:9200
 - <elasticsearch_ip_node_2>:9200
 - <elasticsearch_ip_node_3>:9200
```

```

user.name: ${user.name}
password: ${password}
ssl.certificate.authorities:
 - /etc/filebeat/certs/root-ca.pem
ssl.certificate: "/etc/filebeat/certs/wazuh-server.pem"
ssl.key: "/etc/filebeat/certs/wazuh-server-key.pem"
setup.template.json.enabled: true
setup.template.json.path: '/etc/filebeat/wazuh-template.json'
setup.template.json.name: 'wazuh'
setup.ilm.overwrite: true

```

Figure — `/etc/filebeat/filebeat.yml`

- reduced time for scanning for vulnerabilities to 7 minutes in the `/var/ossec/etc/ossec.conf` file on the Wazuh agent

```

GNU nano 2.2 /var/ossec/etc/ossec.conf
<!-- System inventory -->
<module name="syscollector">
 <disabled>no</disabled>
 <interval>1h</interval>
 <scan_on_start>yes</scan_on_start>
 <hardware>yes</hardware>
</os>yes</os>

root@linux-machine:~# cat /var/ossec/etc/ossec.conf | grep -A5 "syscollector"
 <module name="syscollector">
 <disabled>no</disabled>
 <interval>8m</interval>
 <scan_on_start>yes</scan_on_start>
 <hardware>yes</hardware>
 </os>yes</os>

```

Figures — changing interval on Wazuh Agent configuration

- Go to <https://192.168.30.15/app/vulnerability-detection> and open tab Inventory too look at Vulnerable detections
  - filtered them on my agent name
- Found 3 critical CVEs with `linux-image-6.8.0-106-generic`

agent.name	package.name	package.version	vulnerability.description	vulnerability.severity	vulnerability.id
linux-machine	linux-image-6.8.0-106-generic	6.8.0-106.106	IEEE P802.11-REVme D1.1 through D7.0 allows FragAttacks...	Critical	CVE-2025-27558
linux-machine	linux-image-6.8.0-106-generic	6.8.0-106.106	CWE-502 Deserialization of Untrusted Data at threeventmesh...	Critical	CVE-2024-56180
linux-machine	linux-image-6.8.0-106-generic	6.8.0-106.106	A flaw in netfilter could allow a network-connected attacker L...	Critical	CVE-2021-3773

Figures — 3 critical CVEs

- Removing this package

```

root@linux-machine:~# sudo apt remove linux-image-6.8.0-106-generic
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
and64-microcode bpftc-tools bpfttrace fontconfig-config fonts-dejavu-core
libclang-cpp18 libclang1-18 libcrypt-dev libdbus-glib-1-2 libde265-0 lib
libheif-plugin-libde265 libheif1 libimobiledevice6 libjbig0 libjpeg-tur
libusbmuxd6 libwebp7 libxpm4 linux-headers-generic linux-libc-dev mamp
upower usbmuxd
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
linux-generic linux-image-6.8.0-106-generic linux-image-generic
0 upgraded, 0 newly installed, 3 to remove and 0 not upgraded.
After this operation, 15.1 MB disk space will be freed.
Do you want to continue? [Y/n] Y

```

Figures — removing critical package

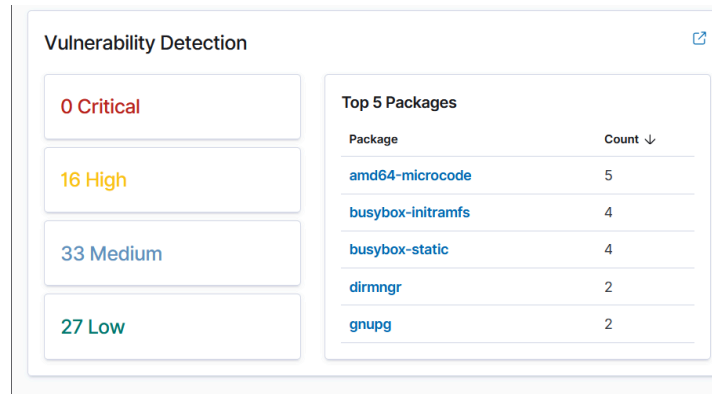


Figure 3c.6 — Remediation applied on Device 3

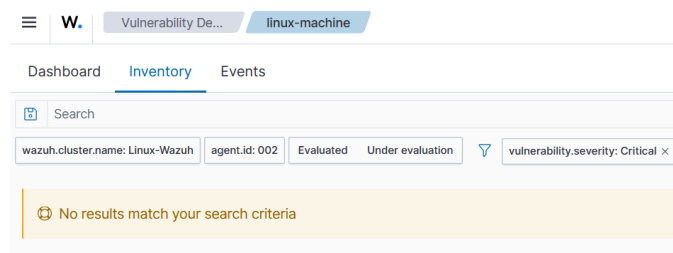


Figure 3c.7 — Updated scan showing vulnerability resolved on all devices

- Then to properly show how vulnerability would like when it is installed on running node, I decided to install this one package and check.

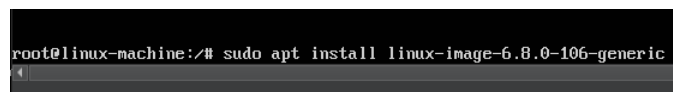


Figure — Installing vulnerable package (image)

- Now inside tab **Events** I can observe such vulnerable packet, that actualy causes a lot of CVEs of different severity.

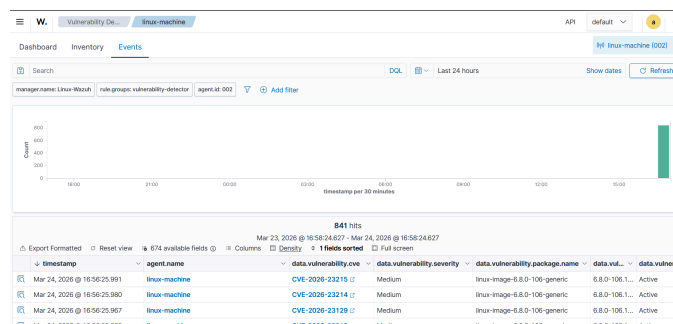


Figure — Events tab with targeted package

- Then I again uninstall it and was waiting to agent run again.

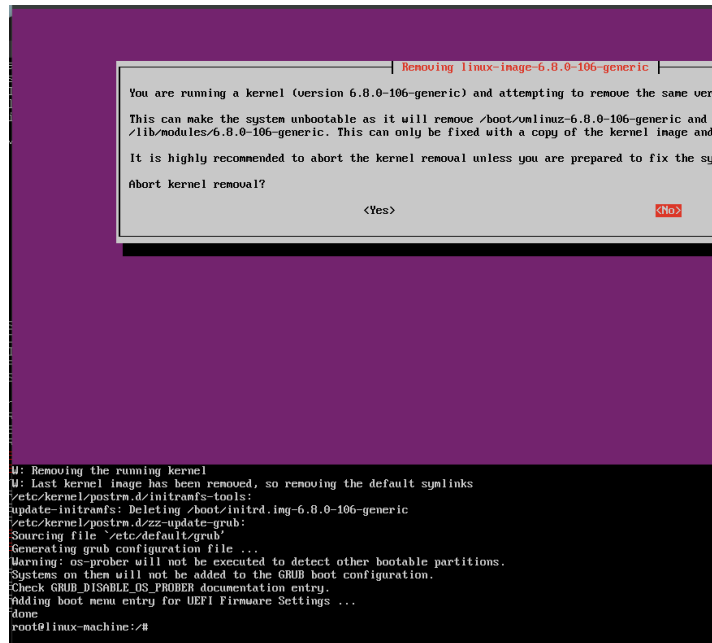


Figure — vulnerable image is uninstalled

- Event now is Resolved:

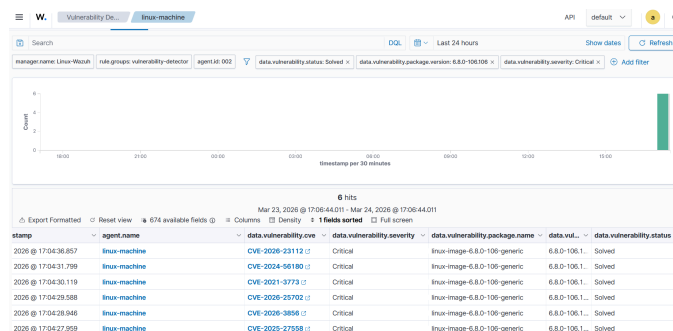


Figure — resolved status on all events with this package.

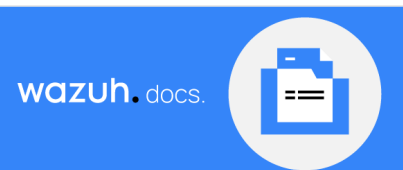
## Part B

### Task 4 — SIEM Integration (*malware*)

Detecting malware using YARA integration - Proof of Concept guide

User manual, installation and configuration guides. Learn how to get the most out of the Wazuh platform.

<https://documentation.wazuh.com/current/proof-of-concept-guide/detect-malware-yara-integration.html>



- NOTE 1:** A sample incident alert should be sent to the owner (you) of the attacked endpoint when alerts are detected for every use case.
- NOTE 2:** Please be careful — THESE ARE REAL MALWARE. Handle in an isolated, sandboxed environment only.

## Configuring linux-machine with agent

- ```
sudo apt update
sudo apt install -y make gcc autoconf libtool libssl-dev pkg-config
sudo curl -LO https://github.com/VirusTotal/yara/archive/v4.5.5.tar.gz
sudo tar -xvzf v4.5.5.tar.gz -C /usr/local/bin/ && rm -f v4.5.5.tar.gz
cd /usr/local/bin/yara-4.5.5/
sudo ./bootstrap.sh && sudo ./configure && sudo make && sudo make install && sudo make check
```

- ```
make[2]: Entering directory '/usr/local/bin/yara-4.5.5'
make[3]: Entering directory '/usr/local/bin/yara-4.5.5'
PASS: test-arena
PASS: test-alignment
PASS: test-atoms
PASS: test-api
PASS: test-rules
PASS: test-pe
PASS: test-elf
PASS: test-version
PASS: test-bitmask
PASS: test-math
PASS: test-stack
PASS: test-re-split
PASS: test-async
PASS: test-string
PASS: test-exception
PASS: test-dotnet

=====
Testsuite summary for yara 4.5.5
=====
TOTAL: 16
PASS: 16
SKIP: 0
XFAIL: 0
FAIL: 0
XPASS: 0
ERROR: 0
=====
make[3]: Leaving directory '/usr/local/bin/yara-4.5.5'
make[2]: Leaving directory '/usr/local/bin/yara-4.5.5'
make[1]: Leaving directory '/usr/local/bin/yara-4.5.5'
root@linux-machine:~# yara
yara: wrong number of arguments
Usage: yara [OPTION]... INAMESPACE:IRULES_FILE... FILE | DIR | PID

Try `--help' for more options
```

- [illegible]

- ```
rule Whispergate_Stage_1 {
    meta:
        description = "Detects first stage payload from WhisperGate"
```

```

    author = "mmuir@cadosecurity.com"
    date = "2022-01-17"
    license = "Apache License 2.0"
    hash = "a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92"
    strings:
        $a = { 31 41 56 4E 4D 36 38 67 6A 36 50 47 50 46 63 4A 75 66 74 4B 41 54 61 34 57 4
C 6E 7A 67 38 66 70 66 76 }
        $b = { 38 42 45 44 43 34 31 31 30 31 32 41 33 33 42 41 33 34 46 34 39 31 33 30 44 3
0 46 31 38 36 39 39 33 43 36 41 33 32 44 41 44 38 39 37 36 46 36 41 35 44 38 32 43 31 45
44 32 33 30 35 34 43 30 35 37 45 43 45 44 35 34 39 36 46 36 35 }
        $c = { 24 31 30 6B 20 76 69 61 20 62 69 74 63 6F 69 6E 20 77 61 6C 6C 65 74 }
        $d = { 74 6F 78 20 49 44 }
    condition:
        uint16(0) == 0x5A4D and all of them
}

rule Whispergate_Stage_2 {
    meta:
        description = "Detects second stage payload from WhisperGate"
        author = "mmuir@cadosecurity.com"
        date = "2022-01-17"
        license = "Apache License 2.0"
        hash = "dcbbae5a1c61dbbbb7dcd6dc5dd1eb1169f5329958d38b58c3fd9384081c9b78"
    strings:
        $a = { 6D 5F 49 6E 74 65 72 63 65 70 74 6F 72 }
        $b = { 6D 5F 62 31 36 65 37 33 65 30 64 61 61 63 34 62 34 33 62 36 35 36 36 39 30 3
1 62 35 34 32 34 63 35 33 }
        $c = { 6D 5F 34 33 37 37 33 32 63 65 65 35 66 35 34 64 37 64 38 34 61 64 64 37 62 6
4 33 30 39 37 64 33 63 61 }
        $d = { 6D 5F 30 64 62 39 37 30 38 63 66 36 34 39 34 30 38 32 39 66 39 61 66 38 37 6
5 64 65 65 64 66 36 30 65 }
        $e = { 6D 5F 65 31 34 33 33 31 36 38 32 30 62 31 34 64 30 33 38 38 61 37 32 37 34 3
4 33 38 65 63 30 37 38 64 }
        $f = { 6D 5F 66 33 31 30 39 30 63 37 31 35 64 65 34 62 30 62 61 62 64 33 31 61 36 3
3 34 31 31 30 34 36 63 38 }
        $g = { 6D 5F 36 31 31 64 31 61 62 63 33 32 66 63 34 66 64 38 61 33 34 65 30 34 34 6
6 39 37 33 34 34 31 64 61 }
        $h = { 6D 5F 37 37 34 62 39 32 31 30 64 39 38 31 34 32 65 62 62 34 34 31 33 35 35 3
9 64 61 61 65 35 61 34 34 }
    condition:
        uint16(0) == 0x5A4D and all of them
}

```

- Create a `yara.sh` script in the `/var/ossec/active-response/bin/` directory. This is necessary for the Wazuh-YARA Active Response scan

```

#!/bin/bash
# Wazuh - Yara active response
# Copyright (C) 2015-2022, Wazuh Inc.
#
# This program is free software; you can redistribute it
# and/or modify it under the terms of the GNU General Public
# License (version 2) as published by the FSF - Free Software

```



```

# Foundation.

#----- Gather parameters -----#

# Extra arguments
read INPUT_JSON
YARA_PATH=$(echo $INPUT_JSON | jq -r .parameters.extra_args[1])
YARA_RULES=$(echo $INPUT_JSON | jq -r .parameters.extra_args[3])
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.syscheck.path)

# Set LOG_FILE path
LOG_FILE="logs/active-responses.log"

size=0
actual_size=$(stat -c %s ${FILENAME})
while [ ${size} -ne ${actual_size} ]; do
    sleep 1
    size=${actual_size}
    actual_size=$(stat -c %s ${FILENAME})
done

#----- Analyze parameters -----#

if [[ ! $YARA_PATH ]] || [[ ! $YARA_RULES ]]
then
    echo "wazuh-yara: ERROR - Yara active response error. Yara path and rules parameters
are mandatory." >> ${LOG_FILE}
    exit 1
fi

#----- Main workflow -----#

# Execute Yara scan on the specified filename
yara_output="$("${YARA_PATH}"/yara -w -r "$YARA_RULES" "$FILENAME")"

if [[ $yara_output != "" ]]
then
    # Iterate every detected rule and append it to the LOG_FILE
    while read -r line; do
        echo "wazuh-yara: INFO - Scan result: $line" >> ${LOG_FILE}
    done <<< "$yara_output"
fi

exit 0;

```

```

root@linux-machine:~# cp /media/yara.sh /var/ossec/active-response/bin/
root@linux-machine:~# ls /var/ossec/active-response/bin/
default-firewall-drop  firewall-drop  host-deny  ipfw  kaspersky.py  pf  restart-wazuh  wazuh-slack
disable-account        firewall-drop  ip-customerblock  kaspersky  npf  restart.sh  route-null  yara.sh
root@linux-machine:~#

```

- Change `yara.sh` file owner to `root:wazuh` and file permissions to `0750`:

```

root@linux-machine:~# chown root:wazuh /var/ossec/active-response/bin/yara.sh
root@linux-machine:~# chmod 750 /var/ossec/active-response/bin/yara.sh
root@linux-machine:~#

```

- Add the following within the `<syscheck>` block of the Wazuh agent `/var/ossec/etc/ossec.conf` configuration file to monitor the directory

```

GNU nano 7.2 /var/ossec/etc/ossec.conf *
<directories realtime="yes">/tmp/yara/malware</directories>

```

- Restart the Wazuh agent

```

root@linux-machine:~# systemctl restart wazuh-agent
root@linux-machine:~# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-25 16:25:37 UTC; 4s ago
     Process: 3070 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 33 (limit: 4601)
      Memory: 25.2M (peak: 32.9M)
        CPU: 7.217s
    CGroup: /system.slice/wazuh-agent.service
            └─3092 /var/ossec/bin/wazuh-execd
              └─3101 /var/ossec/bin/wazuh-agentd
                └─3114 /var/ossec/bin/wazuh-syscheckd
                  └─3130 /var/ossec/bin/wazuh-logcollector
                    └─3182 /var/ossec/bin/wazuh-modulesd

Mar 25 16:25:32 linux-machine systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Mar 25 16:25:32 linux-machine env[3070]: Starting Wazuh v4.14.4...
Mar 25 16:25:32 linux-machine env[3070]: Started wazuh-execd...
Mar 25 16:25:33 linux-machine env[3070]: Started wazuh-agentd...
Mar 25 16:25:34 linux-machine env[3070]: Started wazuh-syscheckd...
Mar 25 16:25:35 linux-machine env[3070]: Started wazuh-logcollector...
Mar 25 16:25:35 linux-machine env[3070]: Started wazuh-modulesd...
Mar 25 16:25:37 linux-machine env[3070]: Completed.
Mar 25 16:25:37 linux-machine systemd[1]: Started wazuh-agent.service - Wazuh agent.
root@linux-machine:~#

```

Configuring Wazuh server

- Add the following rules to the `/var/ossec/etc/rules/local_rules.xml` file. The rules detect FIM events in the monitored directory. They also alert when the YARA integration finds malware. You can modify the rules to detect events from other directories:
 - Copied prev version, then added new groups and copied back

```

<group name="syscheck,">
  <rule id="100300" level="7">
    <if_sid>550</if_sid>
    <field name="file">/tmp/yara/malware/</field>
    <description>File modified in /tmp/yara/malware/ directory.</description>
  </rule>
  <rule id="100301" level="7">
    <if_sid>554</if_sid>
    <field name="file">/tmp/yara/malware/</field>
    <description>File added to /tmp/yara/malware/ directory.</description>
  </rule>
</group>

<group name="yara,">
  <rule id="108000" level="0">
    <decoded_as>yara_decoder</decoded_as>
    <description>Yara grouping rule</description>
  </rule>
  <rule id="108001" level="12">
    <if_sid>108000</if_sid>

```

```

    <match>wazuh-yara: INFO - Scan result: </match>
    <description>File "${(yara_scanned_file)}" is a positive match. Yara rule: ${(yara_rule)}
</description>
</rule>
</group>

```

```

root@Linux-Mazuh:~# cp /media/local_rules.xml /var/ossec/etc/rules/local_rules.xml

```

- Add decoders to local `var/ossec/etc/decoders/local_decoder.xml`, so that YARA stuff will be decoded properly
 - Copied prev version, then added new groups and copied back

```

<decoder name="yara_decoder">
  <prematch>wazuh-yara:</prematch>
</decoder>

<decoder name="yara_decoder1">
  <parent>yara_decoder</parent>
  <regex>wazuh-yara: (\S+) - Scan result: (\S+) (\S+)</regex>
  <order>log_type, yara_rule, yara_scanned_file</order>
</decoder>

```

```

root@Linux-Mazuh:~# cp /var/ossec/etc/decoders/local_decoder.xml /media/local_before_decoders.xml
root@Linux-Mazuh:~# cp /media/local_decoder.xml /var/ossec/etc/decoders/local_decoder.xml

```

- Add the following configuration to the Wazuh server `/var/ossec/etc/ossec.conf` configuration file. This configures the Active Response module to trigger after the rule 100300 and 100301 are fired:
 - Copied prev version, then added new groups and copied back

```

<ossec_config>
  <command>
    <name>yara_linux</name>
    <executable>yara.sh</executable>
    <extra_args>-yara_path /usr/local/bin -yara_rules /tmp/yara/rules/yara_rules.yar</extra_args>
  </command>
  <active-response>
    <disabled>no</disabled>
    <command>yara_linux</command>
    <location>local</location>
    <rules_id>100300,100301</rules_id>
  </active-response>
</ossec_config>

```

```

ossec.conf
local_decoder.xml
ossec.conf
win-setup

ossec.conf
319 <ossec_config>
320   <localfile>
322     <location>journald</location>
323   </localfile>
324
325   <localfile>
326     <log_format>syslog</log_format>
327     <location>/var/ossec/logs/active-responses.
328       log</location>
329   </localfile>
330
331   <localfile>
332     <log_format>syslog</log_format>
333     <location>/var/log/dpkg.log</location>
334   </localfile>
335 </ossec_config>
336
337 <ossec_config>
338   <command>
339     <name>yara_linux</name>
340     <executable>yara.sh</executable>
341     <extra_args>-yara_path /usr/local/bin
342       -yara_rules /tmp/yara/rules/yara_rules.yar</
343       extra_args>
344     <timeout_allowed>no</timeout_allowed>
345   </command>
346
347   <active-response>
348     <disabled>no</disabled>
349     <command>yara_linux</command>
350     <location>local</location>
351     <rules_id>100300,100301</rules_id>
352   </active-response>
353 </ossec_config>

```

- Restart the Wazuh manager to apply the configuration changes:

```

Linux-Wazuh [before malware] [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
root@Linux-Mazuh:~# systemctl restart wazuh-manager
root@Linux-Mazuh:~# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-25 16:34:09 UTC; 22s ago
     Process: 14636 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 219 (limit: 7075)
   Memory: 471.3M (peak: 489.2M)
      CPU: 43.593s
   CGroup: /system.slice/wazuh-manager.service
           └─14699 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─14700 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─14701 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─14704 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─14707 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─14748 /var/ossec/bin/wazuh-authd
                       └─14760 /var/ossec/bin/wazuh-db
                         └─14771 /var/ossec/bin/wazuh-execd
                           └─14803 /var/ossec/bin/wazuh-analysisd
                             └─14813 /var/ossec/bin/wazuh-syscheckd
                               └─14833 /var/ossec/bin/wazuh-remoted
                                 └─14907 /var/ossec/bin/wazuh-logcollector
                                   └─14932 /var/ossec/bin/wazuh-monitord
                                     └─14943 /var/ossec/bin/wazuh-modulesd

Mar 25 16:34:08 Linux-Mazuh env[14636]: Started wazuh-syscheckd...
Mar 25 16:34:08 Linux-Mazuh env[14636]: Started wazuh-remoted...
Mar 25 16:34:06 Linux-Mazuh env[14636]: Started wazuh-logcollector...
Mar 25 16:34:06 Linux-Mazuh env[14636]: Started wazuh-monitord...
Mar 25 16:34:06 Linux-Mazuh env[14941]: 2026/03/25 16:34:06 wazuh-modulesd:router: INFO: Loaded router module.
Mar 25 16:34:06 Linux-Mazuh env[14941]: 2026/03/25 16:34:06 wazuh-modulesd:content-manager: INFO: Loaded conte
Mar 25 16:34:06 Linux-Mazuh env[14941]: 2026/03/25 16:34:06 wazuh-modulesd:inventory-harvester: INFO: Loaded i
Mar 25 16:34:07 Linux-Mazuh env[14636]: Started wazuh-modulesd...
Mar 25 16:34:09 Linux-Mazuh env[14636]: Completed.
Mar 25 16:34:09 Linux-Mazuh systemd[1]: Started wazuh-manager.service - Wazuh manager.
root@Linux-Mazuh:~#

```

Rules (4)

From here you can manage your rules.

| ID | Description | Groups | Level | File | Path |
|--------|--|----------|-------|-----------------|-----------|
| 100000 | File modified in /tmp/yara/malware/ directory. | syscheck | 7 | local_rules.xml | etc/rules |
| 100001 | File added to /tmp/yara/malware/ directory. | syscheck | 7 | local_rules.xml | etc/rules |
| 108000 | Yara grouping rule | yara | 0 | local_rules.xml | etc/rules |
| 108001 | File "yara_scanned_file" is a positive match. Yara rule: yara_rule | yara | 12 | local_rules.xml | etc/rules |

Rows per page: 10

Figure — YARA integration configured in SIEM (Rules)

Decoders (2)

From here you can manage your decoders.

| Name | Program name | Order | File | Path |
|---------------|--|-------|-------------------|--------------|
| yara_decoder | | | local_decoder.xml | etc/decoders |
| yara_decoder1 | log_type, yara_rule, yara_scanned_file | | local_decoder.xml | etc/decoders |

Rows per page: 10

Figure — YARA integration configured in SIEM (Rules)

Installing malware and attacking myself with it

- Now I downloaded malware sample and placed inside `/tmp/yara/malware`

```
root@linux-machine:~# mkdir /tmp/yara/
root@linux-machine:~# mkdir /tmp/yara/malware/
root@linux-machine:~# mkdir /tmp/yara/malware/sus
root@linux-machine:~#
```

```
git clone https://github.com/cado-security/DFIR_Resources_Whispergate
cd DFIR_Resources_Whispergate/Samples
unzip -P "infected" a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92.zip
mv a1* /tmp/yara/malware/sus
```

```
root@linux-machine:~/DFIR_Resources_Whispergate/Samples# cd /tmp/yara/malware/sus/
root@linux-machine:~/tmp/yara/malware/sus# ls
a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92.zip
root@linux-machine:~/tmp/yara/malware/sus# unzip a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92.zip
Archive:  a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92.zip
  inflating: a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92
  inflating: a196c6b8ffcb97ffb276d04f354696e2391311db3841ae16c8c9f56f36a38e92
```

1,502 hits

Mar 24, 2026 @ 15:09:32,988 - Mar 24, 2026 @ 15:09:32,988

| timestamp | agent.name | rule.description | rule.level | rule.id |
|------------------------------|------------|--|------------|---------|
| Mar 24, 2026 @ 05:58:17.8... | linux-host | File "/tmp/yara/malware/sus" is a positive match. Yara rule: APT_HKTL_Wisper_WhisperGate_Jan22_1_RID331C | 12 | 108001 |
| Mar 24, 2026 @ 05:58:17.8... | linux-host | File "/tmp/yara/malware/sus" is a positive match. Yara rule: Whispergate_Stage_1 | 12 | 108001 |
| Mar 24, 2026 @ 05:58:15.2... | linux-host | File added to /tmp/yara/malware/ directory. | 7 | 100301 |

Figure — WhisperGate sample detected (no execution)

Bonus

- ★ Bonus tasks — complete for extra credit.

Bonus 1 — Automate Incident Alert Notifications

Automate the sample incident alerts sent in the use cases section using any platform of choice (e.g. email via Postfix, Slack webhook, PagerDuty, Shuffle SOAR).

Describe the automation platform chosen and how it is connected to the SIEM alert pipeline.

```
# Example: Wazuh custom integration for Slack notification
# /var/ossec/integrations/custom-slack
curl -X POST -H 'Content-type: application/json' \
  --data '{"text": "SIEM Alert: '$ALERT_DESCRIPTION'"}' \
  https://hooks.slack.com/services/YOUR/WEBHOOK/URL
```



Figure B1.1 — Automated alert notification platform configured



Figure B1.2 — Automated incident alert received (email / Slack / etc.)

References

1. Install Mikrotik chr on Virtual box [[image vdi](#)] [[guide](#)].
2. Windows QEMU has VNC console type and Windows 7 w/ IE10. Default credentials: IEUser / Passw0rd!
3. Windows 11 installation from GNS3: <https://www.gns3.com/marketplace/appliances/windows-11-dev-env>
4. Official: <https://www.microsoft.com/ru-ru/software-download>



Fix - To Continue Enter An Admin Username And Password In Windows 11/ 10 || 'Yes' Button Missing

Fix - To Continue Enter An Admin Username And Password In Windows 11/ 10 || 'Yes' Button Missing

Fix - To Continue Enter An Admin Username And Password In Windows 11/ 10 || 'Yes' Button Missing.

If you like the tutorial, then please support my YouTube channel by subscribing it and clicking the bell icon for

https://www.youtube.com/watch?v=_fMYBwWJ2P4

